

CI /CD

CI/CD is a process of automating code integration, build creation, and testing.

- GitLab CI/CD is a popular tool for implementing CI/CD pipelines.
- It involves steps like code commit, pull request, code merge, build creation, and test case execution.

- Tools like Selenium and JUnit are used for uploading test cases.
- Tools like BlackTurk and SonarQube are used for security vulnerability checks and static code analysis.
- Builds are uploaded to a build repository like SonarTypeNexus and JFrog Artifactory.
- Integration is done using tools like Jenkins, AWS CodePipeline, and GitLab CI.

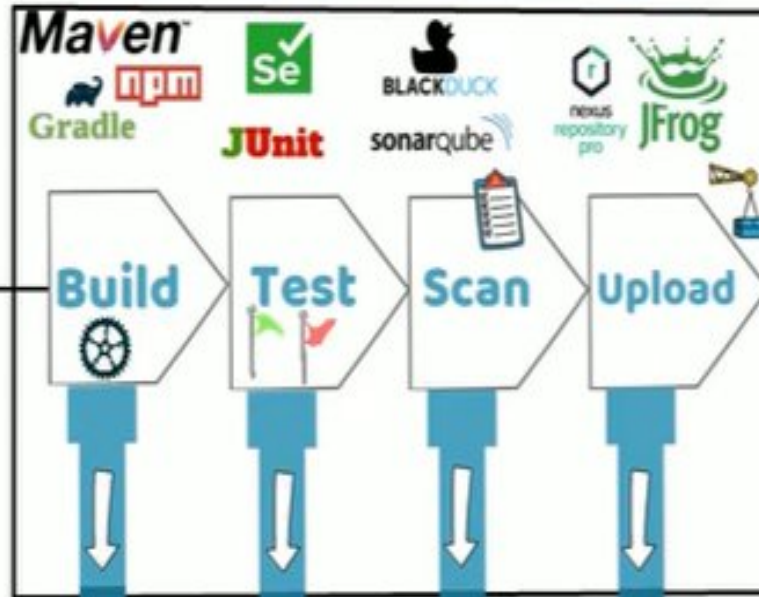
- Deployment is triggered for the dev server first and then for the UAD server after passing sanity checks.
- Fraud deployment is triggered after passing the sanity check on the UAD server.

- Continuous delivery involves continuous feedback and monitoring at all stages of the deployment lifecycle.
- A release cycle includes build, test, scan, upload, development deployment, UAT deployment, and production deployment.

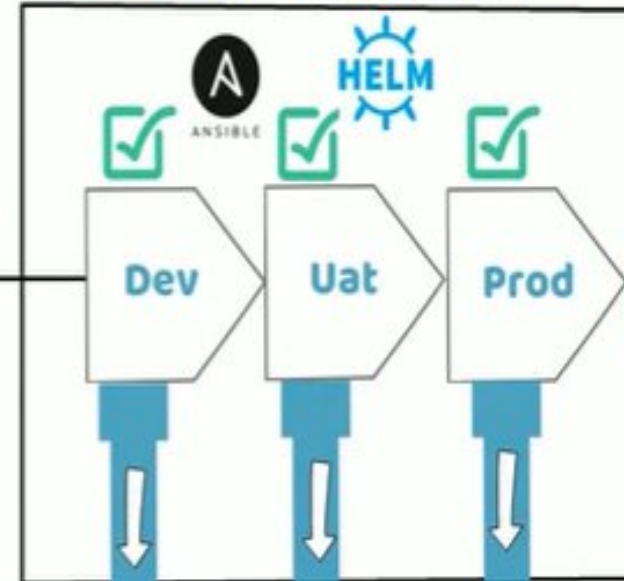


AWS CodePipeline

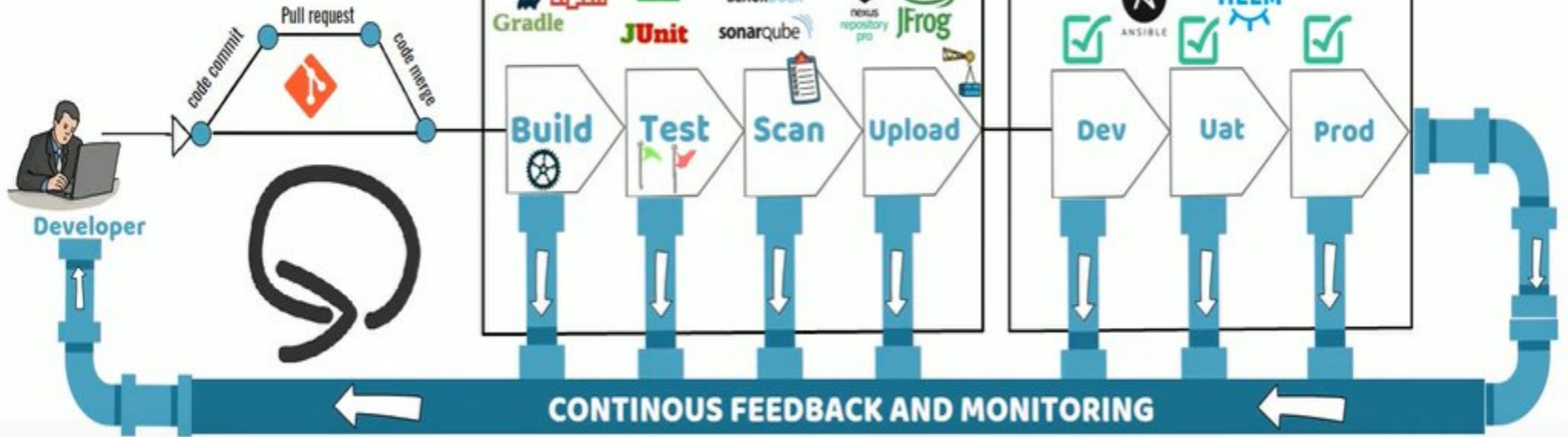
COUNTINUOUS INTEGRATION



COUNTINUOUS DEPLOYMENT/DELIVERY



CONTINUOUS FEEDBACK AND MONITORING



What is Jenkins?

Jenkins is an open-source automation server.

It helps automate the parts of software development related to building, testing, and deploying, facilitating Continuous Integration and Continuous Delivery (CI/CD).

Why Use Jenkins?

- Automates code builds and tests
- Integrates with Git, Docker, Maven, Kubernetes, etc.
- Supports pipelines for end-to-end automation
- Has 1800+ plugins for customization
- Works across different OS platforms

Term	Description
Job/Project	A task to be executed (e.g., build a project)
Build	A single run of a job
Node	A machine where Jenkins runs jobs (Master or Agent/Slave)
Pipeline	A series of steps (scripted or declarative) to define CI/CD workflow
Plugin	Add-on that gives Jenkins extra functionality (e.g., Git plugin, Docker plugin)

Jenkins Setup Basics

1.Install Jenkins

- Download from jenkins.io
- Run with Java (java -jar jenkins.war) or install via Docker

2.Access Jenkins Dashboard

- Go to <http://localhost:8080> in your browser
- Enter the admin password (found in the Jenkins home folder)

Install Suggested Plugins

- **During setup, Jenkins will recommend a few plugins—install them.**

Create First Admin User

- **Set up username, password, and email.**

Creating Your First Job

- Click “New Item”
- Enter a name
- Choose “Freestyle project”
- Configure:
 - Source Code Management (Git)
 - Build Trigger (e.g., Poll SCM or on Push)
 - Build (e.g., Execute Shell or Invoke Maven)
- Click “Save” and then “Build Now”

Think of a Job as:

"Tell Jenkins what to do and when to do it."
Jobs are the core unit of automation in Jenkins.

- Automating Repetitive Work
- Enabling Continuous Integration (CI)
- Setting Up Continuous Delivery (CD)
- Custom Workflows and Pipelines
- Integrations

End-to-End CI/CD Pipeline
Implementation using Jenkins,
GitHub, Maven, and Docker
Add Required Plugins

Pipeline Graph Analysis	Success
Pipeline: REST API	Success
Pipeline: Stage View	Success
Dev Tools Symbols API	Success
Javadoc	Success
JSch dependency	Installing
Maven Integration	Pending
Authentication Tokens API	Pending
Docker Commons	Pending
SSH server	Pending
Git server	Pending
Docker Pipeline	Pending
HTML Publisher	Pending
Role-based Authorization Strategy	Pending
SSH Agent	Pending
Commons Compress API	Pending
Pipeline Utility Steps	Pending
Loading plugin extensions	Pending

```
mkdir neww
cd new
mkdir -p src/main/java/com/example
notepad src/main/java/com/example/Application.java
package com.example;
```

```
import org.springframework.boot.*;
import org.springframework.boot.autoconfigure.*;
import org.springframework.web.bind.annotation.*;
```

```
@SpringBootApplication
@RestController
public class Application {

    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }

    @GetMapping("/")
    public String home() {
        return "CI/CD Working!";
    }
}
```


notepad Dockerfile

FROM openjdk:17

COPY target/*.jar app.jar

ENTRYPOINT ["java","-jar","/app.jar"]

notepad Jenkinsfile

```
pipeline {
```

```
    agent any
```

```
    tools {
```

```
        maven 'Maven'
```

```
    }
```

```
    stages {
```

```
        stage('Build') {
```

```
            steps {
```

```
                bat 'mvn clean package'
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

Create pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>app</artifactId>
  <version>1.0</version>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.0</version>
  </parent>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
```

git init

git add .

git commit -m "added project files"

git branch -M main

git remote add origin <https://github.com/preetkaur18/newww.git>

git push -u origin main

VERIFY ON GITHUB

Jenkinsfile

pom.xml

Dockerfile

src folder

Apply restart with Jenkins

- Go to **Jenkins Dashboard**
- ci-cd-project
- Click on **Configure**
- Scroll down in configuration page
- Find the **Pipeline Section**
- Set **Definition** as:
Pipeline script from SCM
- Set **SCM** as:
Git
- Enter Repository URL:
<https://github.com/preetkaur18/newww.git>
- Enter Branch:
*/main
- Enter Script Path:
Jenkinsfile
- Click **Save**
- **Apply build now**

What is a Jenkins Pipeline?

A pipeline is a set of automated steps that Jenkins follows to build, test, and deploy your application. Think of it as a scripted flowchart for your DevOps process.

Why Use a Pipeline?

- Automate the entire CI/CD process
- Make builds repeatable and version-controlled
- Run complex workflows (parallel stages, conditional logic, etc.)
- Use code to define build steps (called "Pipeline as Code")

Two Types of Pipelines

Type

Description

Declarative

Easier to read and write.
Structured and recommended
for beginners.

Scripted

More flexible but uses Groovy
scripting, best for advanced
users.

What Are Plugins in Jenkins?

Plugins are add-ons that extend Jenkins' core functionality.
They help Jenkins integrate with tools like:

- GitHub, GitLab
- Docker
- Slack
- Maven, Gradle
- JIRA
- Kubernetes
- And much more!

Basically, if Jenkins needs to do anything beyond basic build automation, there's probably a plugin for it.

Why Are Plugins Important?

Without plugins, Jenkins is just a basic automation server. Plugins let you:

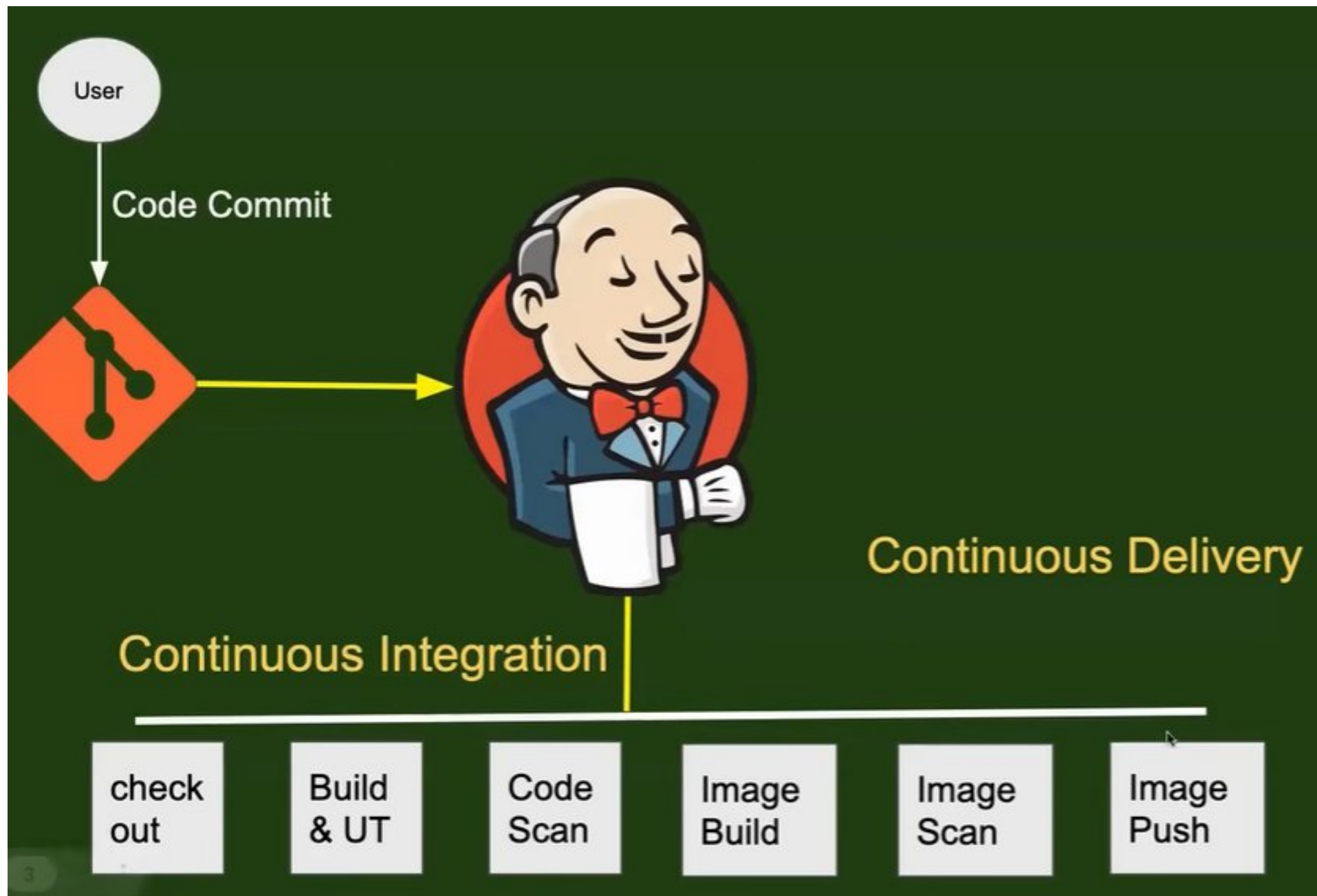
- Connect to source code repositories (e.g., Git plugin)
- Add build tools (e.g., Maven, Gradle plugin)
- Send notifications (e.g., Email, Slack)
- Use containers (e.g., Docker plugin)
- Define complex pipelines (Pipeline plugin)

Plugin Name	What It Does
Git Plugin	Lets Jenkins pull code from Git
Pipeline Plugin	Enables pipeline-as-code using <code>Jenkinsfile</code>
Blue Ocean	A modern UI for Jenkins Pipelines
Docker Pipeline	Lets you run Docker containers inside pipelines
Slack Notification	Sends build status messages to Slack
Email Extension	Sends custom emails on build success/failure
JUnit Plugin	Publishes test results
Credentials Plugin	Securely stores passwords, tokens, and SSH keys

How to Install a Plugin

- Go to Jenkins Dashboard
- Click on Manage Jenkins
- Click on Manage Plugins
- Search in the Available tab
- Click Install without restart or Download now and install after restart

Too many plugins = performance issues or conflicts. Only install what you need.



Overview of Jenkins

- Jenkins is an open-source automation server designed to assist developers in automating the software development lifecycle.
- Initially created for automating the build process for Java, Jenkins now supports a variety of programming languages and technologies.
- By automating repetitive tasks—such as building, testing, and deployment—Jenkins allows developers to concentrate on coding.

Key Features of Jenkins

- Continuous Integration (CI) and Continuous Delivery (CD) are core features, enabling automatic building, testing, and deployment of code changes.
- Jenkins utilizes a rich ecosystem of plugins, which extend its functionality and support automation for diverse tasks in the development workflow.
- Jenkins pipelines allow developers to define the entire build, test, and deployment process as code, facilitating version control and collaboration.

How Jenkins Enhances Workflow

- By automating routine processes, Jenkins reduces the potential for human error and saves time, streamlining the software development process.
- Integrations with other tools in the software development ecosystem make Jenkins adaptable to existing workflows, enhancing productivity.
- Developers can write pipeline scripts using Groovy, a domain-specific language, enabling the definition of complex workflows and task automation.

Importance for Developers and DevOps Engineers

- Jenkins is considered an essential tool in the software development field, providing significant benefits in CI/CD capabilities.
- The automation features of Jenkins enable teams to catch errors and bugs early in the development cycle, improving software quality.
- With its robust automation capabilities and wide-ranging integrations, Jenkins empowers developers to focus on creating high-quality code.

Dashboard ▾

 New Item

 People

 Build History

 Manage Jenkins

 My Views

 Lockable Resources

 New View

Build Queue

^

No builds in the queue.

Build Executor Status

^

1 Idle

2 Idle

[add description](#)

Welcome to Jenkins!

This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.

Start building your software project

Create a job →

Set up a distributed build

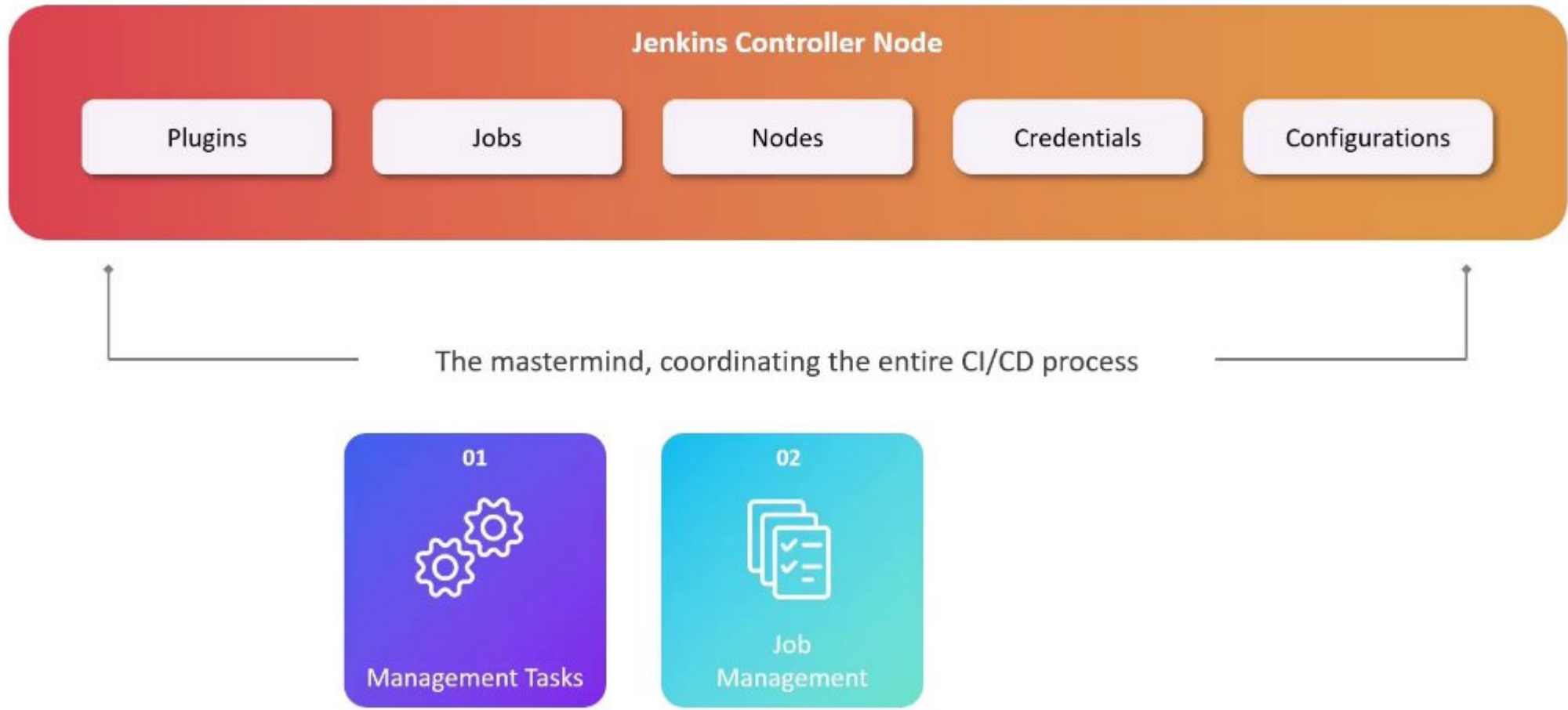
Set up an agent →

Configure a cloud →

Learn more about distributed builds ↗

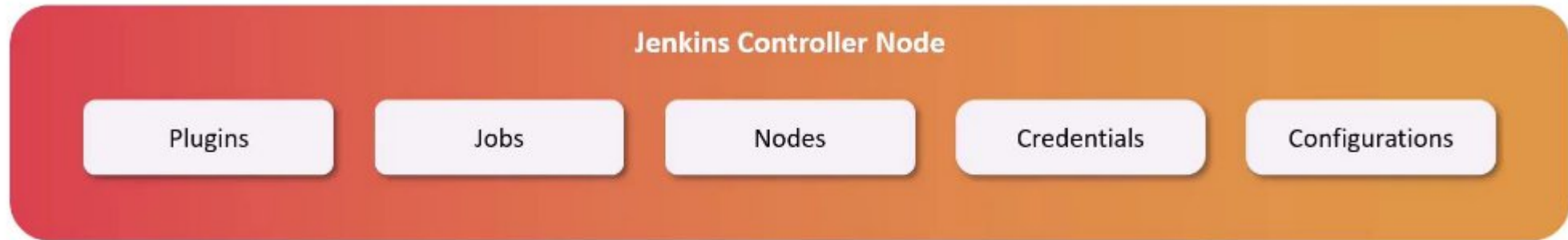


Jenkins Architecture





Jenkins Architecture



01

Basic Deployment

Controller and worker node are the same



Limited to
smaller project

02

Advanced Deployment

Separate controller and worker nodes



Prevents
Accidental
Damage



Boosts
Performance



Scalability for
Growth

Jenkins Architecture

